MDB200

Intro to Atlas Search & Vector Search

MongoDB Optimization and Performance

Release: 20260220

Topics we cover

What is Atlas Search

How to set up Atlas Search Index

What is Atlas Vector Search

How to set up Atlas Vector Search Index

Additional Atlas Vector use cases (RAG)

Search nodes

Atlas Search

What is "Atlas" Search?

Atlas Search is an addition to the hosted MongoDB database allowing the creation of indexes to support Search using Apache Lucene, the world's leading text search technology.

Search allows you to make additional searchable indexes in a different format optimised for relevance searching.

Although this is something that can be built and configured yourself with a self-hosted MongoDB, it's complicated to do well. Users often have to install, configure, and pay for ElasticSearch or SOLR to do this.

With Atlas, it's as simple as specifying what data you want to search and the way you intend to search it. The indexes are then built and maintained for you in the background.

And as a developer, you can now perform relevance-ranked searching (and more) on behalf of your users.

Best of all, it's all integrated, managed, and has no additional licensing costs or splitting of support between multiple organisations.

What do we mean by search?

"With a database query - you know exactly what you want.
When you use search you are open to suggestions." - unknown

```
db.movies.find({ title : "Raising Arizona"})
```
Title IS EXACTLY "Raising Arizona"

```
db.movies.find({ title: /Raising Arizona/})
```
Title CONTAINS the phrase "Raising Arizona."

```
db.movies.aggregate([{$search : { text : { query : "Raising Arizona", path: "title" }}}])
```
Title mentions some variations of "Raising," "Arizona," or both

MongoDB is a database. As a database, it can safely perform storage and retrieval of records with guarantees of safety of interaction between multiple users and durability of writes and predictable interaction between them.

It's a primary data store and a system of record.

Atlas is MongoDB, hosted on public cloud, with day to day maintenance done for you by teams inside MongoDB - allowing you to focus on using the database, rather then running it.

So what is Atlas Search?

The "Read" in CRUD, which includes find() and aggregate() for MongoDB, identifies all documents exactly matching a specific criteria.

This match is boolean. A document matches - or does not - and the notion of matching is very exact.

Search, on the other hand, accepts that some things match better than others and allows us to order by how well they match.

This is most notable in filtering by text fields. Rather than an exact match, we can find where fields contain some of the query terms, or even similar query terms.

Scoring and Relevance

Search: Javascript Book

Title	Score
Javascript the Definitive Guide	75
Javascript the Good Parts	60
The Beginners Guide to Node.js	35
Angular for Dummies	25
Introducing MongoDB	10

When we use Atlas Search, results are sorted not by a specific value in the data, like a normal database sort, but by those that match our query the best.

To do this, each result is assigned a "score." How exactly that score is assigned is something we have a lot of control over in Atlas Search.

We will be looking at score in more detail later on.

This score is said to denote the "relevance" of a result, so by sorting and getting the highest score first, we can see the results our user will most want to see, the "most relevant."

Accessing Atlas Search

Loading Sample Data in Atlas

Connect to a free Atlas cluster from a mongo shell
Add a small number of documents

```
> use searchtraining

> db.example.drop()

//N.B. There are deliberate spelling errors below

> db.example.insertMany([
{ text: "This is a sentance and I can search for it using
 just one word" },
{ text: "This is a a different sentence, and I can search
 for it using  one or two words" },
{ text: " Whether it is one word or two words could make
a difference to the results"}
])


> db.example.find({})
```


In order to best see the functionality of Atlas Search, we are going to add a tiny dataset. This makes it easier to understand our results in the context of every record in the collection.

Atlas Search works very quickly on any size of data but by using just a few documents, we can see what is does and does not return and how it scores them relatively.

Run the following in a MongoDB shell to create the sample data:

```
use searchtraining
db.example.drop()
db.example.insertMany([
{_id:1, text: "This is a sentance and I can search for it using just one word
" },
{_id:2, text: "This is a a different sentence, and I can search for it using
 one or two words" },
{_id:3, text: " Whether it is one word or two words could make a difference t
o the results"}
])
```

You can then see this in the collections tab in Atlas.

View Documents in Atlas

Overview | Real Time | Metrics | Collections | Atlas Search | Query Insights | Performance Advisor | Online Archive | Cmd

DATABASES: 11 COLLECTIONS: 36

PREVIEW New Data Explorer | VISUALIZE YOUR DATA | REFRESH

+ Create Database

Search Namespaces

- sample_airbnb
- sample_analytics
- sample_geospatial
- sample_guides
- sample_mflix
- sample_restaurants
- sample_supplies
- sample_training
- sample_weatherdata
- searchtraining
  - example

searchtraining.example

STORAGE SIZE: 36KB LOGICAL DATA SIZE: 315B TOTAL DOCUMENTS: 3 INDEXES TOTAL SIZE: 20KB

Find | Indexes | Schema Anti-Patterns 0 | Aggregation | Search Indexes

Generate queries from natural language in Compass | INSERT DOCUMENT

Filter | Type a query: { field: 'value' } | Reset | Apply | Options

QUERY RESULTS: 1-3 OF 3

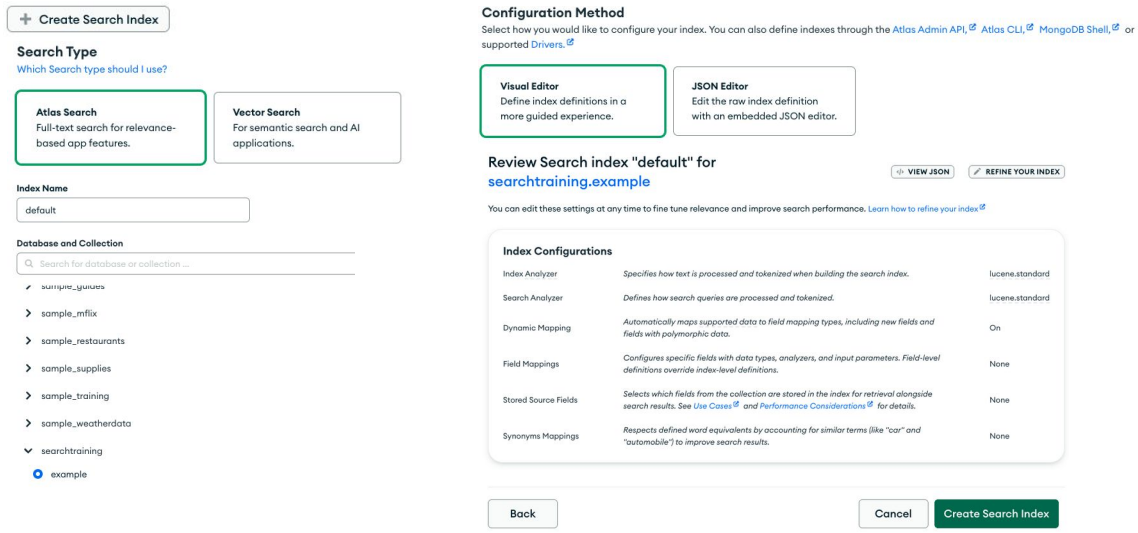
```
_id: ObjectId('685b31b95995e1470b69e328')
text : "This is a sentance and I can search for it using just one word"

_id: ObjectId('685b31b95995e1470b69e329')
text : "This is a a different sentence, and I can search for it using  one or …"

_id: ObjectId('685b31b95995e1470b69e32a')
```


Inside Atlas, use "Browse Collections" to take a look at the three documents you added, complete with deliberate spelling mistakes. They are in a database called searchtraining and a collection called example.

Creating a Default Search Index



Now that we have inserted some documents and we can connect and see them in the GUI and via a shell, we need to tell Atlas we want to create a Search index for this data.

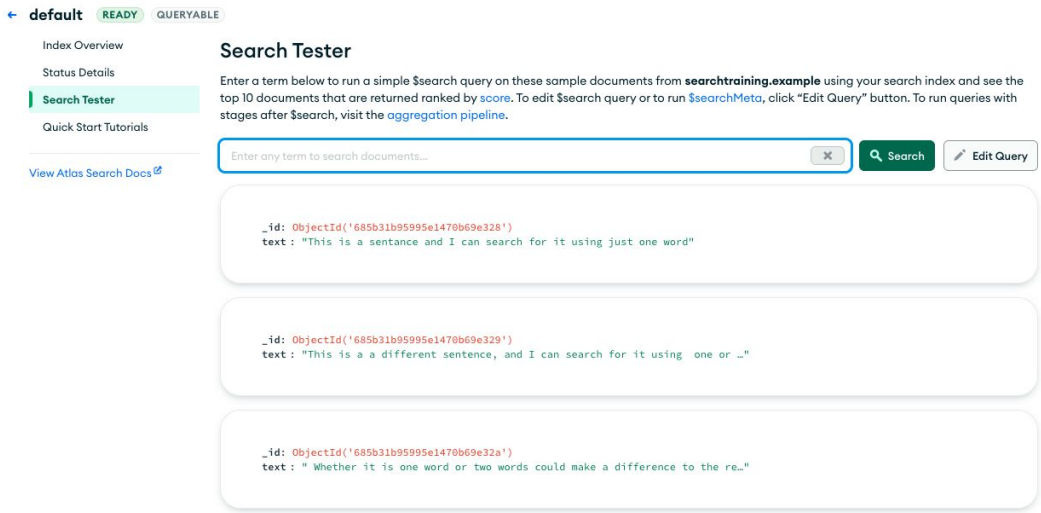
To do this, we click the Search tab and then "Create Search Index." As this is our first time, it takes us through a simple path.

We can use either a GUI editor (The Visual Editor) to configure our index or edit a JSON configuration file. For this first example, we will choose Visual Editor and click Next.

In the next screen, will keep the index name as "default." The index Atlas search uses this index if nothing is explicitly specified.

We need to unfold the **searchtraining** database and select the **example** collection.
We will accept all the default settings for now.
We are using the "Standard" text analyzer, so strings will be split into searchable values based on spaces and punctuations but will not know any specifics of the language.
We are using the "dynamic" mapping modes which means all number, text, and date fields will be indexed, including those in arrays or objects.
Click "Create Search Index" and then close the window that pops up. The index will take a few minutes to create.
We know it's done when it says "3 (100%) indexed of 3 total."

Performing a Simple Text Search



Once the index is built, then the "Query" button will be available and take us to the "Search Tester."

"Search Tester" is a very basic search application/GUI we can use to see how our search index would look or behave in an application.

Now we can type in a query, just as we would in any other search engine, and see matching results, scores, and highlighted terms.

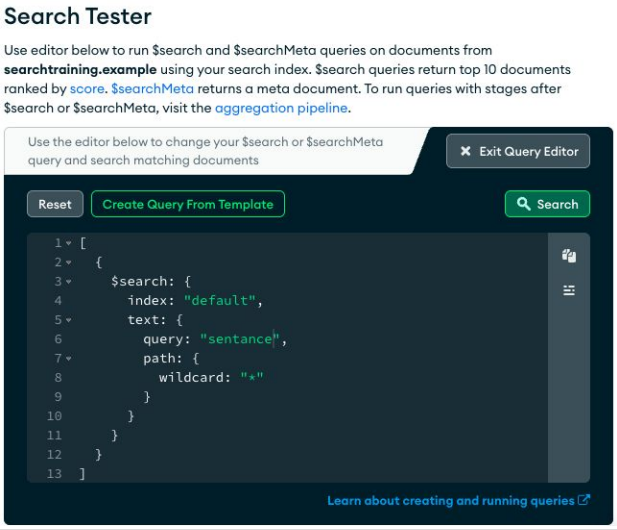
Try searching for "**one**" - we can see documents that contain the word one in the context (where the word occurs) and also a score as to how good this was.

Try searching for "**one two**" - note we have a higher score for things with both one and two, but we still find the line with just one.

Try a search for "**sentance**" - note we do not find "sentence." Atlas Search can do this, but this GUI does not enable fuzzy searching. We cover that later.

Click "View Query Syntax" to see what it's doing. Note it's searching the index "default." It's a simple text query and the path (fields to search) is set to the wildcard:"*" which means every field in this index.

Searching from Code




```
[{ $search : {
     index: "default",
     text : {
        query : "sentance",
        path : {
          wildcard : "*"
     }}}}]
```


It's nice that the Atlas GUI gives us a specific search GUI, but that's just a basic, completed app. How can we perform this query in our code?
We can see that using the MongoDB shell, which is itself a REPL allowing us to run code and see the results. Using the connect details in your Atlas cluster, open up a MongoDB shell connected to your cluster. It will be a command like this one but with your unique cluster id and username.

```
mongosh "mongodb+srv://cluster1.kshqv.mongodb.net" --username myuser
```

Now, in the search tester in the GUI, with the intentionally misspelled word *sentance* typed in the box, click "View Query Syntax" and copy to clipboard.
$search is an **Aggregation** stage. It must be the first stage in the aggregation. If you are unfamiliar with aggregation, it's a set of transformative steps you can apply to documents before they are returned from the database.

We could add more stages like more matching, sorting, limiting the number of results, or rewriting the shape of the output by adding more pipeline stages.
We specify an **index**. Otherwise, it uses 'default.' We are specifying a **text** operation (find values in text field). There are other types of search. We will talk more about this later.

In the the mongodb shell type:

```
use searchtraining
query = <PASTE VALUE FROM GUI or SLIDE HERE>
db.example.aggregate(query)
```

We can now see how easy it is to run a search from the shell or from any other driver.
We will be using the Mongo shell to demonstrate functionality in this class as we need to pass various options, and the Search Test UI has those hard coded for what it does.

Viewing the Result Relevance

We can see the score by using $set to add an additional output field

```
> let query = [ {    $search: { index: 'default',
      text: {  query: 'sentance',
        path: {   'wildcard': '*' }
      }   }  },
  { $set: { score : { $meta: "searchScore" }}}
]



Atlas [primary] searchtraining> db.example.aggregate(quer
y)
[
  {
    _id: ObjectId("..."),
    text: 'This is a sentance and I can search for it
using just one word',
    score: 0.46227604150772095
  }
]
```

14

The number in the search tester GUI, the score, shows how relevant our results are. In this case, relevance is judged on how often the value appears and also how many different fields it appears in.
We can also see on the list which fields it matched in. If a term is shown appearing in the same field multiple times, those will be different instances of the field as part of an array of objects.

To see this scoring information, we have to project a special metadata field called searchScore. We can do this with an additional pipeline stage where we add the score metadata to the output. We use $set to add a field to the aggregation output.

```
use searchtraining
let query = [ {    $search: { index: 'default',
      text: {  query: 'sentance',
        path: {   'wildcard': '*' }
      }   }  },
  { $set: { score : { $meta: "searchScore" }}}
]
db.example.aggregate(query)
```

Now we can see a score for this result. If we had multiple matching results, they would be ordered by this score.

Exercise

You searched for the word "sentance" and output it with the score in the shell

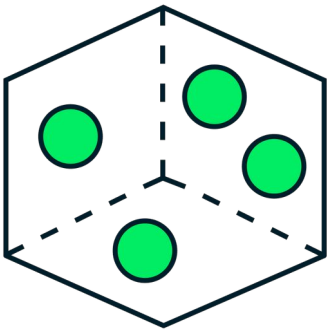
Try to search for "one" and then "one two" and output those with the score in the shell

Atlas Vector Search

What is Atlas Vector Search?

Atlas Vector search allows:

- Semantic search
- Question and Answer systems
- Image search
- Synonym generation
- Feature extraction
- Recommendations & Relevance Scoring
- Improve AI systems
- Retrieval Augmented Generation (RAG)
- etc.



In Atlas you can create and store vector embeddings with machine learning models like OpenAI for retrieval augmented generation (RAG), semantic search, recommendation engines, dynamic personalization, and other use cases.
This new functionality allows you to Build semantic search* and AI-powered applications.

Semantic search uses context clues to determine the meaning of a word across a dataset of millions of examples. Semantic search also identifies what other words can be used in similar contexts.
For example: finding a "sweater" with the query "warm clothing" or "how can I keep my body warm in the winter?".
We can use vector search to build and enhance Q&A systems and generate Large language model (LLM) memory increasing the context for AI applications, providing up to date information, and reducing AI hallucinations.

Atlas Vector Search allows searching through data based on semantic meaning captured in vectors, whereas **Atlas Search** allows for keyword search i.e. based on the actual text and any defined synonym mappings.

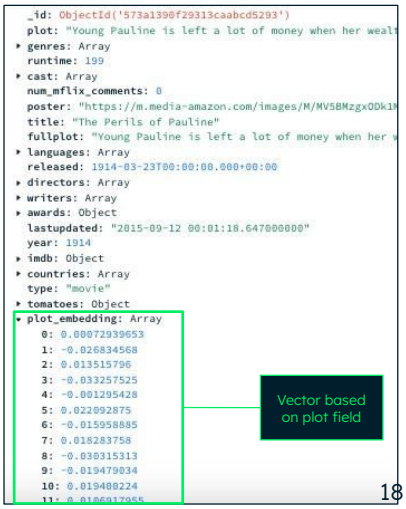
What is Atlas Vector Search?

Search unstructured data based on **semantic meaning** captured in *vectors*

Vectors are numeric representations of the data in an array of doubles that is embedded with the document

Use with `$vectorSearch` aggregation stage

```
db.embedded_movies.aggregate([{ "$vectorSearch": {
      "index": "vector_index",
      "path": "plot_embedding",
      "queryVector": [0.024552748,-0.015502235,...,-0.033900633],
      "numCandidates": 150,
      "limit": 10
    } }]
```



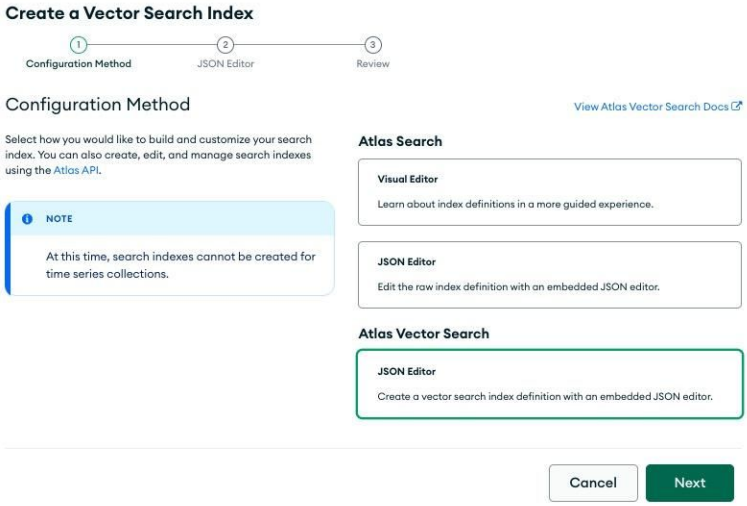
Vector embeddings are integrated with application data in the same document and seamlessly indexed for semantic queries, enabling you to build easier and faster.
It uses the Hierarchical Navigable Small Worlds algorithm to perform the semantic search. You can use Atlas Vector Search support for approximate Nearest Neighbor (aNN) queries to search for results similar to a queryVector

Note: In the sample data set, under the sample_mflix DB, there is already a movies collection with the embedded array (vector) for the plot field.
The plot_embedding field contains embeddings created using OpenAI's text-embedding-ada-002 embedding model.
See more info here: https://platform.openai.com/docs/guides/embeddings/what-are-embeddings

Creating the Index

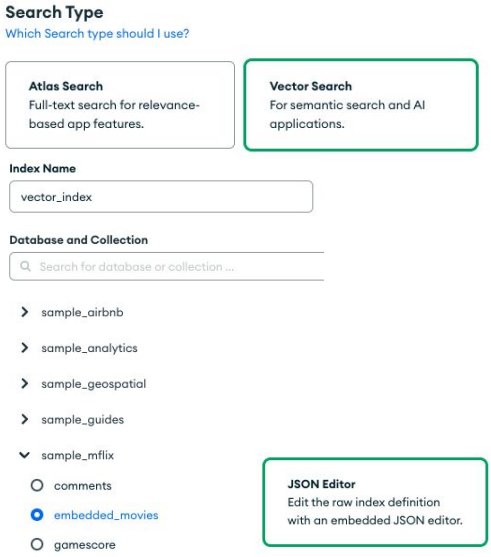
In Atlas,

1. Go to Atlas Search
2. Create an Index
3. Select Atlas Vector Search - JSON Editor



Creating the Index

```
{
  "fields": [
      {
        "type": "vector",
        "path":"plot_embedding",
         "numDimensions": 1536,
         "similarity": "euclidean"
       }
    ]
  }
```



The "similarity" field can be set to "euclidean", "cosine", or "dotProduct"

Limitation:
- You can't index fields inside arrays of documents or fields inside arrays of objects
- Atlas Vector Search supports embeddings up to 2048 dimensions

The index definition in the example for the sample_mfilx.embedded_movies collection dynamically indexes all the dynamically indexable fields in the collection and statically indexes plot_embedding field as the Vector type.
The plot_embedding field contains embeddings created using OpenAI's text-embedding-ada-002 embeddings model.
The index definition specifies 1536 vector dimensions and measures similarity using euclidean.

Note: You can also optionally index fields for pre-filtering the data against which you want to run your Atlas Vector Search query.

Using vector search

```
db.embedded_movies.aggregate([

 { "$vectorSearch": {
      "exact": false
       "index": "vector_index",
       "path": "plot_embedding",
      "queryVector": [0.024552748,...],
       "numCandidates": 150,
       "limit": 10
     }

  },

  { "$project": {
       "_id": 0,
       "plot": 1,
       "title": 1,
       "score": { $meta: "vectorSearchScore" }
     }
  }
])
```

```
  {
    plot: 'A reporter, learning of time
 know by averting upcoming disasters.',
    title: 'Thrill Seekers',
    score: 0.7892671227455139
  },
  {
    plot: 'At the age of 21, Tim discov
n life. His decision to make his world
ght think.',
    title: 'About Time',
    score: 0.7843604683876038
  },
  {
    plot: 'Hoping to alter the events o
uture, where he finds humankind divided
    title: 'The Time Machine',
    score: 0.7801066637039185
  },
  {
    plot: "After using his mother's new
 up in a children's crusade where he co
    title: 'Crusade in Jeans',
    score: 0.7789170742034912
  },
  {
    plot: 'An officer for a security ag
tician who has a tie to his past.',
    title: 'Timecop',
    score: 0.7771612405776978
  },
  {
    plot: 'A time-travel experiment in
y wrong thrusting one of the project sc
    title: 'A.P.E.X.',
    score: 0.7730885744094849
  },
  {
    plot: "Agent J travels in time to M.
ent K and changing history.",
    title: 'Men in Black 3',
    score: 0.7712380886077881
  },
  {
    plot: 'Bound by a shared destiny, a
bark on a mission to unearth the secret
ory.',
    title: 'Tomorrowland',
    score: 0.7669923901557922
  },
  {
    plot: 'With the help of his uncle,
    title: 'Love Story 2050',
    score: 0.7649372816885816
  },
  {
    plot: 'A dimension-traveling wizard
 his magic. With his home world on the
    title: 'The Portal',
    score: 0.7640786170959473
  }
```

21

See a basic example in the documentation:
https://www.mongodb.com/docs/atlas/atlas-vector-search/vector-search-stage/
Note: This can take a few minutes as the vector is long

Execute this aggregation command in mongosh using the Instruqt lab.
We have stored a sample embedding into a variable called `vector_embedding_demo`
To set the variable you need to run the `query_embedding.js` file - see steps below.
In the lab terminal:

1. Connect to the atlas cluster
2. Once connected, run:

```
use sample_mflix

load("query_embedding.js")

vector_embedding_demo

const pipeline = [ { $vectorSearch: { index: "vector_index", path:
"plot_embedding", queryVector: vector_embedding_demo, numCandidates:
100, limit: 10 } }, { $project: { title: 1, plot: 1, score: { $meta:
"vectorSearchScore" } } }];

db.embedded_movies.aggregate(pipeline)
```

Change the limit parameter to see the different results returned.

If the "exact" parameter is set to 'true', it enables ENN (Exact Nearest Neighbour). If "exact" is set to false or omitted, search will be ANN (Approximate Nearest Neighbor), and the numCandidates field must be present.

Exercise: Find similar movies with Vector search

Titanic was a great movie, let's get some recommendations on related movies based on this movie plot using vector search

1. Use the `sample_mflix.embedded_movies` collection
2. Find the movie using the title filter: `{title: "Titanic"}`
3. Use vector search, with the Titanic's `plot_embedding` as the `queryVector` to find movies with similar plots.


```
_id: ObjectId('573a139af29313caabcefb1d')
plot : "The story of the 1912 sinking of the larg
genres : Array (3)
runtime : 173
cast : Array (4)
poster : "https://m.media-amazon.com/images/M/MV5
title : "Titanic"
fullplot : "The plot focuses on the romances of t
languages : Array (1)
released : 1996-11-17T00:00:00.000+00:00
rated : "PG-13"
awards : Object
lastupdated : "2015-08-30 00:47:02.163000000"
year : 1996
imdb : Object
countries : Array (2)
type : "series"
tomatoes : Object
num_mflix_comments : 0
plot_embedding : Array (1536)
```

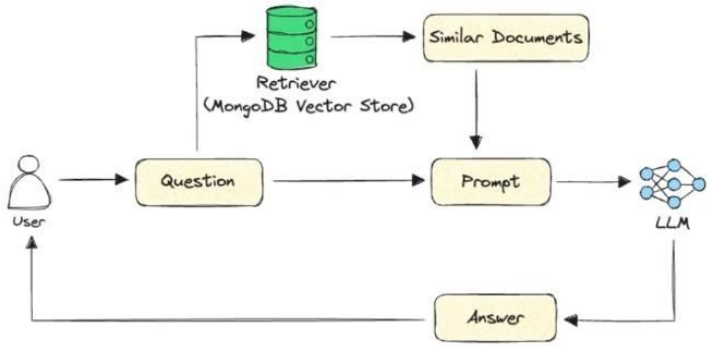

Answers in the appendix
Try different titles to test the results

RAG with Vector Search

RAG: Retrieval Augmented Generation

Prompt the LLM with relevant information

Reduces hallucinations

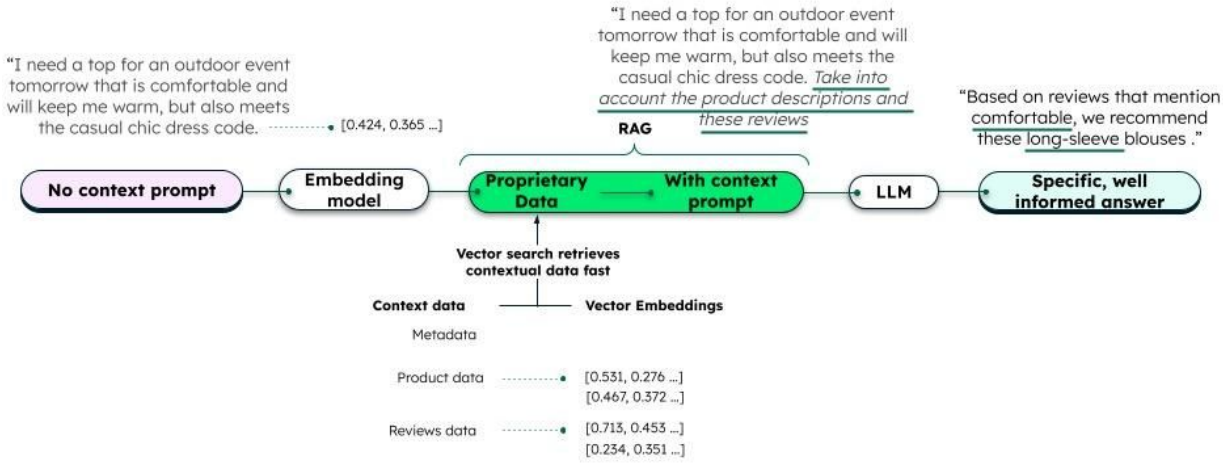


Large Language Models (LLMs) have revolutionized the field of natural language processing, but they do come with certain limitations: hallucinations, stale data, no access to users' local data, etc.

RAG uses vector search to retrieve relevant documents based on the input query. It then provides these retrieved documents as context to the LLM to help generate a more informed and accurate response.

Instead of generating responses purely from patterns learned during training, RAG uses those relevant retrieved documents to help generate a more informed and accurate response.

RAG Basic Architecture



A basic retrieval-augmented generation architecture consists of these main components:

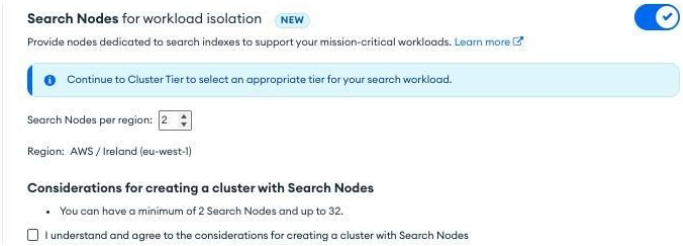
- A pre-trained LLM: The LLM is responsible for generating text, images, audio, and video.
- Vector search (or semantic search): The retrieval system is responsible for finding relevant information from a knowledge base that is external to the LLM. There are a variety of general purpose databases or single-purpose vector databases to choose from that can store vector embeddings and run approximate nearest neighbor search queries against them. Vector search is core to being able to precisely augment proprietary knowledge provided to a general purpose LLM.
- Vector embeddings: Sometimes referred to simply as "vectors" or "embeddings", vector embeddings are essentially numerical representations capturing the semantic, or underlying meaning of a piece of data. Generally speaking, they're an array of floats, where each float represents a single dimension of the numerical representations.
- Orchestration: The fusion mechanism is responsible for combining the output of the LLM with the information from the retrieval system to generate the final output.

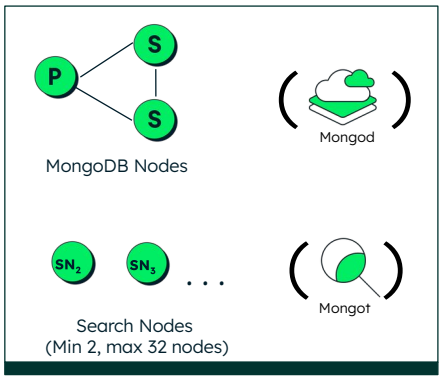
Atlas Search Nodes

Dedicated Search nodes

Search nodes allow:

- Resource Optimization and Flexibility
- Better Performance
- Higher Availability





Atlas Vector Search also takes advantage of our new Search Nodes dedicated architecture, enabling better optimization for the right level of resourcing for specific workload needs.
Search Nodes provide dedicated infrastructure for Atlas Search and Vector Search workloads, allowing you to optimize compute resources and fully scale search needs independent of the database.
Atlas supports deploying search nodes for Atlas Search and Vector Search separately:

- Resource Optimization and Flexibility: Confidently handle and scale demanding Search workloads
- Better performance : 40% - 60% decrease in query time for many complex queries
- Higher Availability: Workload isolation avoids resource contention between database and search

https://www.mongodb.com/docs/atlas/cluster-config/multi-cloud-distribution/#std-label-configure-search-nodes

Quiz Time!

#1. In which of the following scenarios would you use Atlas Search vs MongoDB query?

A	In a database of song lyrics, find a song you heard on the radio
B	In a database of bank statements, find the name of a restaurant you went to
C	In a database of hotels, find one with a pool and archery as activities
D	In a database of customers, find the highest spenders and what they buy
E	Display pop-up adverts on a website

Answer in the next slide.

#1. In which of the following scenarios would you use Atlas Search vs MongoDB query?

A. In a database of song lyrics, find a song you heard on the radio

B. In a database of bank statements, find the name of a restaurant you went to

C. In a database of hotels, find one with a pool and archery as activities

D. In a database of customers, find the highest spenders and what they buy

E. Display pop-up adverts on a website

If you need to run a query where you know exactly what you're looking for with an additional sort in mind, then using a query is the way to go as option D suggests. All other options can be handed by Atlas Search, including finding names of restaurant (implies a like) and even curating ads based on previous searches.

#2. Vector Search enables which 3 features?

A	Semantic searches	B	Structured queries	C	Recommendations & Scoring
D	Image searches	E	Temporal analysis		

Answer in the next slide.

#2. Vector Search enables which 3 features?

A Semantic searches

B Structured queries

C Recommendations & Scoring

D Image searches

E Temporal analysis

Vector search doesn't support temporal analysis or structured queries but can be used for A, C, D

Recap

Atlas Search is an Atlas feature that creates indexes to support Search using Apache Lucene.

Search results are scored to rate how well they match the search.

Atlas search includes many search operators that can be combined. It also includes autocomplete, synonyms, and highlight capabilities.

Atlas Vector Search allows searching through data based on semantic meaning captured in vectors

Congratulations on completing MDB200. You now have basic knowledge of MongoDB indexes and how to troubleshoot performance issues. In the next module, you will learn how to prepare MongoDB for scalability and reliability in production. Register to attend now if you have not already done so.

https://mdb.link/mdb100
https://mdb.link/mdb200
https://mdb.link/mdb300

Appendix

Exercise Vector Search - Solution

Use the plot_embedding from Titanic's movie to find similar movies

```
use sample_mflix;
my_movie = db.embedded_movies.findOne({ title: "Titanic" });

pipeline = [
 {"$vectorSearch": {
     "exact": false,
     "index": "vector_index",
     "path": "plot_embedding",
     "queryVector": my_movie.plot_embedding,
     "numCandidates": 150,
     "limit": 10
   }
 },
 {"$project": {
     "_id": 0,
     "plot": 1,
     "title": 1,
     "score": { $meta: "vectorSearchScore" }
   }
 }
]
print("Similar movies to: "+ my_movie.title);
movies = db.embedded_movies.aggregate(pipeline); // you can add .skip(1) to remove Titanic
```



```
// Now try searching for movies based on directors!
// Below should be same movie as {directors: "George Lucas", title: /^Star
Wars: Episode IV/ }
my_movie = db.embedded_movies.findOne({ directors: "George Lucas" });

//pipeline is a variable so can be updated in-place.
pipeline[0]["$vectorSearch"].queryVector = my_movie.plot_embedding;

print("Similar movies to: "+ my_movie.title);
db.embedded_movies.aggregate(pipeline).skip(1);
```